

Class10 Structural Bioinformatics

Qiman A17912409

Intro to the RCSB and PDB Statistics

Read a CSV file of current PDB stats obtained from <https://www.rcsb.org/stats/summary>

```
pdb <- read.csv("data.csv")
pdb
```

	Molecular.Type	X.ray	EM	NMR	Integrative	Multiple.methods
1	Protein (only)	180,758	23,111	12,813	348	229
2	Protein/Oligosaccharide	10,488	3,741	34	8	11
3	Protein/NA	9,205	6,751	287	26	8
4	Nucleic acid (only)	3,154	250	1,578	3	15
5	Other	178	27	35	4	0
6	Oligosaccharide (only)	11	0	6	0	1

	Neutron	Other	Total
1	84	32	217,375
2	1	0	14,283
3	0	0	16,277
4	3	1	5,004
5	0	0	244
6	0	4	22

Q1: What percentage of structures in the PDB are solved by X-Ray and Electron Microscopy.

```
pdb$X.ray
```

```
[1] "180,758" "10,488" "9,205" "3,154" "178" "11"
```

```
pdb$EM
```

```
[1] "23,111" "3,741" "6,751" "250" "27" "0"
```

This printout above `pb$X.ray` is “character” not “numeric”. I can’t do math with it so to fix it:

```
#changing the printout to numeric using gsub()

pdb <- read.csv("data.csv", check.names = FALSE)

pdb$`X-ray` <- as.numeric(gsub(",", "", pdb$`X-ray`))
pdb$EM <- as.numeric(gsub(",", "", pdb$EM))
pdb$Total <- as.numeric(gsub(",", "", pdb$Total))

#Now that the printout is numeric I can calculate the percentages

xray_percent <- sum(pdb$`X-ray`) / sum(pdb$Total) * 100
em_percent <- sum(pdb$EM) / sum(pdb$Total) * 100

xray_percent
```

```
[1] 80.48577
```

```
em_percent
```

```
[1] 13.38046
```

80.5% are solved by X-Ray crystallography while 13.4% are solved by Electron Microscopy

Q2: What proportion of structures in the PDB are protein?

```
protein <- 217375
total <- 253205

protein/total
```

```
[1] 0.8584941
```

85.8% of the structures in the PDB are protein(only)

The total number of protein sequences in uniprot is 202,556,314

217375/202556314 *100

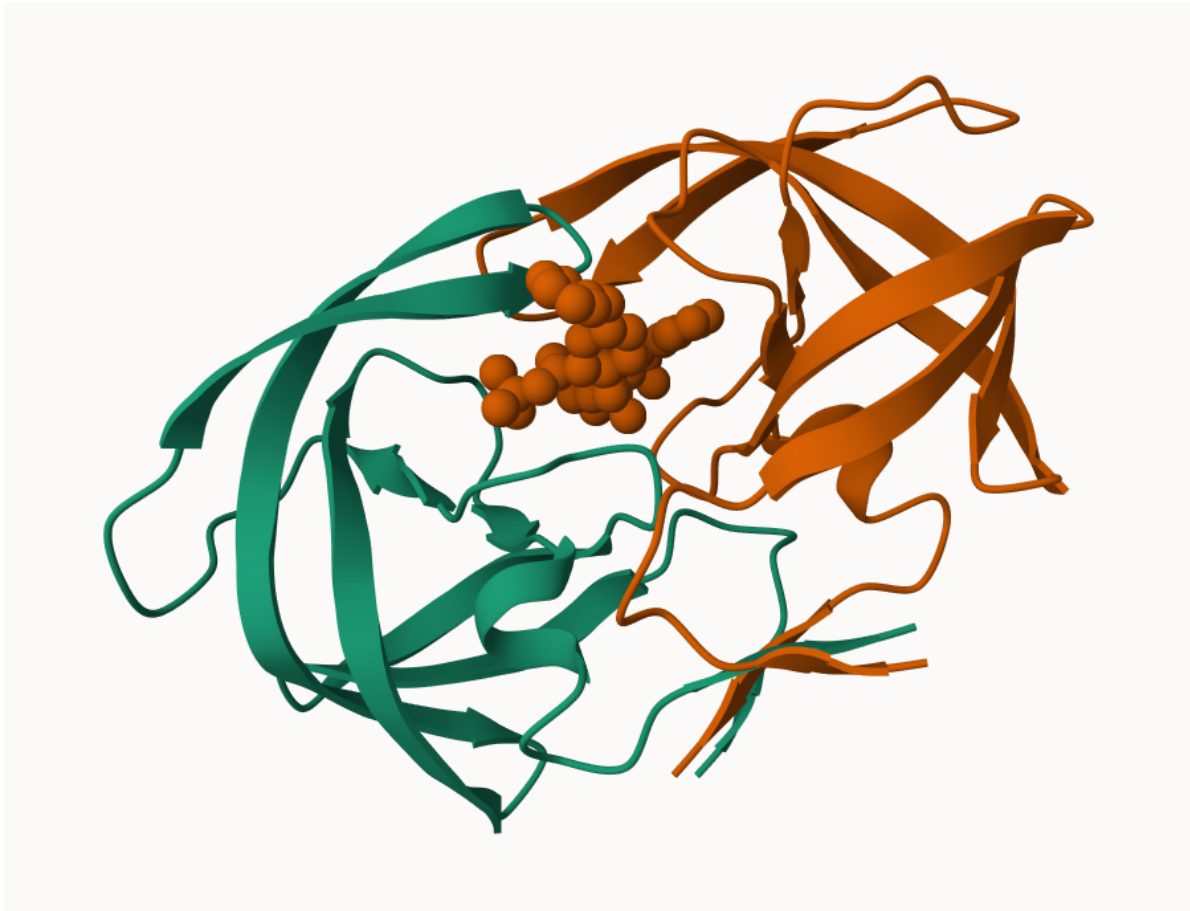
[1] 0.1073158

We have a very small structural coverage of known proteins (~.1%). Most structures we know (~80%) came from one method (X-ray crystallography)

Q3: Type HIV in the PDB website search box on the home page and determine how many HIV-1 protease structures are in the current PDB?

There are 1,147 HIV-1 protease structures in the current PDB.

Visualizing the HIV-1 protease structure



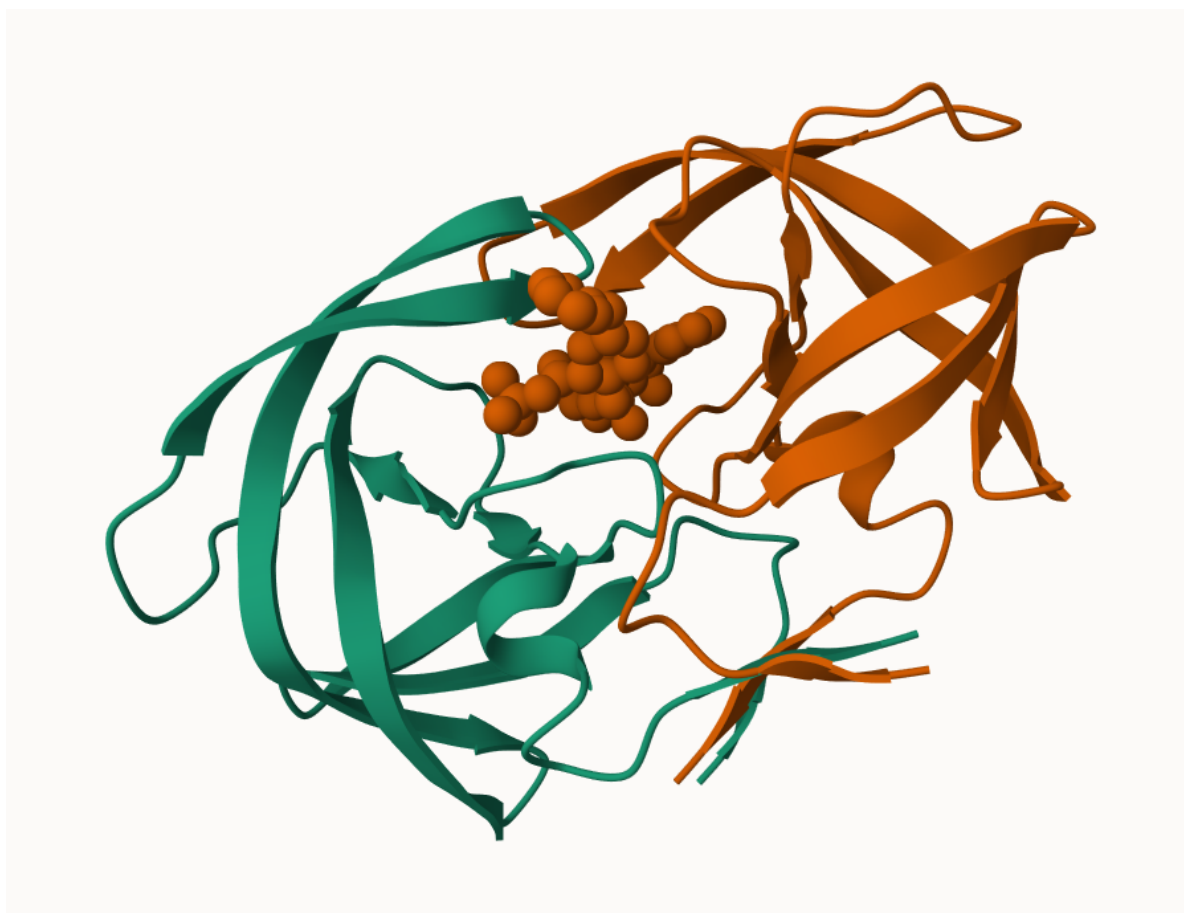
Q4: Water molecules normally have 3 atoms. Why do we see just one atom per water molecule in this structure?

We see one atom per water molecules because in most X-ray crystal structures the hydrogen atoms of water are not visible. The PDB structures usually only include the oxygen atom of each water molecule.

Q5: There is a critical “conserved” water molecule in the binding site. Can you identify this water molecule? What residue number does this water molecule have

HOH 308

Q6: Generate and save a figure clearly showing the two distinct chains of HIV-protease along with the ligand. You might also consider showing the catalytic residues ASP 25 in each chain and the critical water (we recommend “Ball & Stick” for these side-chains). Add this figure to your Quarto document.



Introduction to Bio3D in R

Bio3D is an R package for structural bioinformatics. Features include the ability to read, write and analyze biomolecular structure, sequence and dynamic trajectory data.

In your existing Rmarkdown document load the Bio3D package by typing in a new code chunk:

```
library(bio3d)
pdb <- read.pdb("1hsg")
```

Note: Accessing on-line PDB file

```
pdb
```

Call: read.pdb(file = "1hsg")

```
Total Models#: 1
Total Atoms#: 1686, XYZs#: 5058 Chains#: 2 (values: A B)

Protein Atoms#: 1514 (residues/Calpha atoms#: 198)
Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)

Non-protein/nucleic Atoms#: 172 (residues: 128)
Non-protein/nucleic resid values: [ HOH (127), MK1 (1) ]
```

```
Protein sequence:
PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD
QILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFPQITLWQRPLVTIKIGGQLKE
ALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTP
VNIIGRNLLTQIGCTLNF
```

```
+ attr: atom, xyz, seqres, helix, sheet,
      calpha, remark, call
```

Q7: How many amino acid residues are there in this pdb object?

There are 198 amino acid residues in this pdb object.

Q8: Name one of the two non-protein residues?

One non-protein residue is HOH.

Q9: How many protein chains are in this structure?

There are 2 protein chains in this structure.

We can use the Bio3D partner package, `bio3dview`, to generate quick interactive molecular visualizations. To install the development version of `bio3dview` from GitHub, along with the related `NGLViewer` package use:

```
##install.packages("remotes")
##remotes::install_github("bioboot/bio3dview")
##install.packages("NGLViewer")
```

```
library(bio3dview)
library(NGLViewer)
```

```
##view.pdb(pdb) |>
## setSpin()
```

```
# Select the important ASP 25 residue
#sele <- atom.select(pdb, resno=25)

# and highlight them in spacefill representation
#view.pdb(pdb, cols=c("navy","teal"),
#  highlight = sele,
#  highlight.style = "spacefill") |>
# setRock()
```

Let's read a new PDB structure of Adenylate Kinase and perform Normal mode analysis:

```
adk <- read.pdb("6s36")
```

Note: Accessing on-line PDB file
PDB has ALT records, taking A only, `rm.alt=TRUE`

```
adk
```

Call: `read.pdb(file = "6s36")`

Total Models#: 1
Total Atoms#: 1898, XYZs#: 5694 Chains#: 1 (values: A)


```
Protein Atoms#: 1654 (residues/Calpha atoms#: 214)
Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)
```

```
Non-protein/nucleic Atoms#: 244 (residues: 244)
Non-protein/nucleic resid values: [ CL (3), HOH (238), MG (2), NA (1) ]
```

Protein sequence:

```
MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLRAAVKSGSELGKQAKDIMDAGKLV
DELVIALVKERIAQEDCRNGFLLDGFPRTIPQADAMKEAGINVDYVLEFDVPDELIVDKI
VGRRVHAPSGRVYHVKFNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQM TAPLIG
YYSKEAEAGNTKYAKVDGTPVAEVRADLEKILG
```

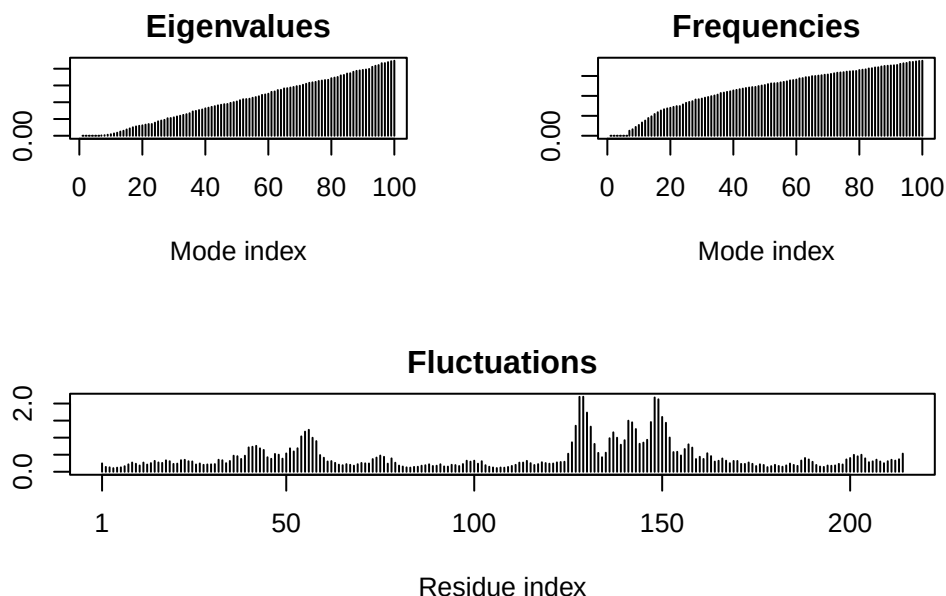
```
+ attr: atom, xyz, seqres, helix, sheet,
      calpha, remark, call
```

Normal mode analysis (NMA) is a structural bioinformatics method to predict protein flexibility and potential functional motions (a.k.a. conformational changes).

```
# Perform flexibility prediction
m <- nma(adk)
```

```
Building Hessian...      Done in 0.04 seconds.
Diagonalizing Hessian... Done in 0.2 seconds.
```

```
plot(m)
```



To view a “movie” of these predicted motions we can generate a molecular “trajectory” with the `mktrj()` function.

```
mktrj(m, file="adk_m7.pdb")
```

Alternatively, for a quicker display you can use the `view.nma()` function from the `bio3dview` package mentioned previously:

```
#view.nma(m, pdb=adk)
```

Comparative structure analysis of Adenylate Kinase

The goal of this section is to perform principal component analysis (PCA) on the complete collection of Adenylate kinase structures in the protein data-bank (PDB).

```
##install.packages("BiocManager")
##BiocManager::install("msa")
```

Q10. Which of the packages above is found only on BioConductor and not CRAN?

`msa` is only found on Bioconductor and not CRAN.

Q11. Which of the above packages is not found on BioConductor or CRAN?:

bio3dview is not found on Bioconductor or CRAN.

Q12. True or False? Functions from the pak package can be used to install packages from GitHub and BitBucket?

True, functions from the pak package can be used to install packages from Github and BitBucket.

Below we perform a blast search of the PDB database to identify related structures to our query Adenylate kinase (ADK) sequence.

```
library(bio3d)
aa <- get.seq("1ake_A")
```

Warning in get.seq("1ake_A"): Removing existing file: seqs.fasta

Fetching... Please wait. Done.

aa

```

      1      .      .      .      .      .      .      60
pdb|1AKE|A  MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAAVKSGSELGKQAKDIMDAGKLVT
      1      .      .      .      .      .      .      60

      61      .      .      .      .      .      .      120
pdb|1AKE|A  DELVIALVKERIAQEDCRNGFLLDGFPRTIPQADAMKEAGINVDYVLEFDVPDELIVDRI
      61      .      .      .      .      .      .      120

     121      .      .      .      .      .      .      180
pdb|1AKE|A  VGRRVHAPSGRVYHVKNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTAPLIG
     121      .      .      .      .      .      .      180

     181      .      .      .      214
pdb|1AKE|A  YYSKEAEAGNTKYAKVDGTPVAEVRADLEKILG
     181      .      .      .      214
```

Call:

```
read.fasta(file = outfile)
```

Class:

```
fasta
```

Alignment dimensions:

1 sequence rows; 214 position columns (214 non-gap, 0 gap)

+ attr: id, ali, call

Q13. How many amino acids are in this sequence, i.e. how long is this sequence?

There are 214 amino acids in this sequence.

```
hits <- NULL
hits$ pdb.id <- c('1AKE_A', '6S36_A', '6RZE_A', '3HPR_A', '1E4V_A', '5EJE_A', '1E4Y_A', '3X2S_A', '6H...

# Download releated PDB files
files <- get.pdb(hits$ pdb.id, path="pdbs", split=TRUE, gzip=TRUE)
```

```
Warning in get.pdb(hits$ pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1AKE.pdb exists. Skipping download
```

```
Warning in get.pdb(hits$ pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6S36.pdb exists. Skipping download
```

```
Warning in get.pdb(hits$ pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6RZE.pdb exists. Skipping download
```

```
Warning in get.pdb(hits$ pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3HPR.pdb exists. Skipping download
```

```
Warning in get.pdb(hits$ pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4V.pdb exists. Skipping download
```

```
Warning in get.pdb(hits$ pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/5EJE.pdb exists. Skipping download
```

```
Warning in get.pdb(hits$ pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4Y.pdb exists. Skipping download
```

```
Warning in get.pdb(hits$ pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3X2S.pdb exists. Skipping download
```

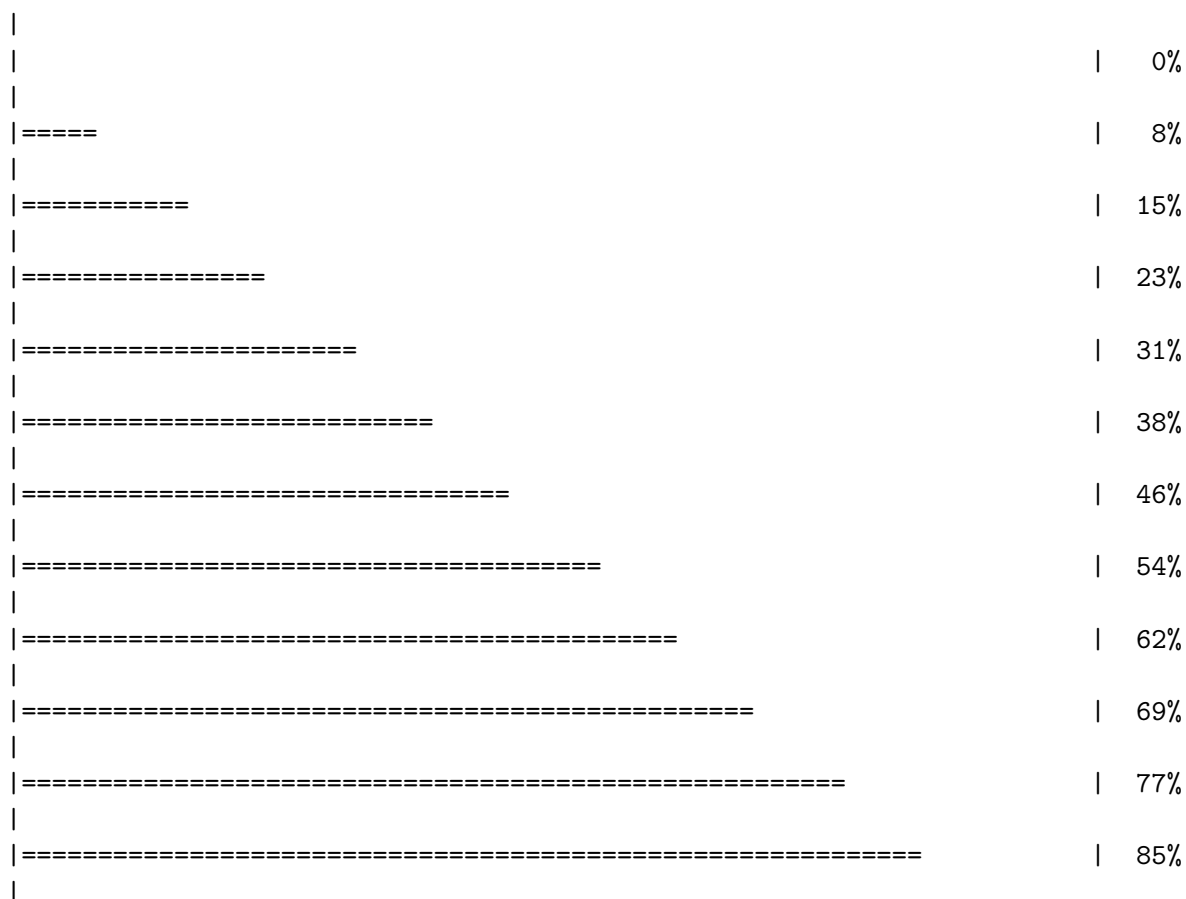
Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAP.pdb exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAM.pdb exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4K46.pdb exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3GMT.pdb exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4PZL.pdb exists. Skipping download



```
|=====| 92%
|
|=====| 100%
```

Next we will use the `pdbaln()` function to align and also optionally fit (i.e. superpose) the identified PDB structures.

```
# Align related PDBs
pdbs <- pdbaln(files, fit = TRUE, exefile="msa")
```

Reading PDB files:

```
pdbs/split_chain/1AKE_A.pdb
pdbs/split_chain/6S36_A.pdb
pdbs/split_chain/6RZE_A.pdb
pdbs/split_chain/3HPR_A.pdb
pdbs/split_chain/1E4V_A.pdb
pdbs/split_chain/5EJE_A.pdb
pdbs/split_chain/1E4Y_A.pdb
pdbs/split_chain/3X2S_A.pdb
pdbs/split_chain/6HAP_A.pdb
pdbs/split_chain/6HAM_A.pdb
pdbs/split_chain/4K46_A.pdb
pdbs/split_chain/3GMT_A.pdb
pdbs/split_chain/4PZL_A.pdb
  PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..  PDB has ALT records, taking A only, rm.alt=TRUE
.... PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
...
```

Extracting sequences

```
pdb/seq: 1   name: pdbs/split_chain/1AKE_A.pdb
  PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 2   name: pdbs/split_chain/6S36_A.pdb
  PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 3   name: pdbs/split_chain/6RZE_A.pdb
  PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 4   name: pdbs/split_chain/3HPR_A.pdb
```

```

PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 5   name: pdbs/split_chain/1E4V_A.pdb
pdb/seq: 6   name: pdbs/split_chain/5EJE_A.pdb
PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 7   name: pdbs/split_chain/1E4Y_A.pdb
pdb/seq: 8   name: pdbs/split_chain/3X2S_A.pdb
pdb/seq: 9   name: pdbs/split_chain/6HAP_A.pdb
pdb/seq: 10  name: pdbs/split_chain/6HAM_A.pdb
PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 11  name: pdbs/split_chain/4K46_A.pdb
PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 12  name: pdbs/split_chain/3GMT_A.pdb
pdb/seq: 13  name: pdbs/split_chain/4PZL_A.pdb

```

We can view our superposed results with the new `bio3dview view()` function:

```

library(bio3dview)

##view.pdbs(pdbs)

```

The function `pdb.annotate()` provides a convenient way of annotating the PDB files we have collected. Below we use the function to annotate each structure to its source species. This will come in handy when annotating plots later on:

```

# Vector containing PDB database codes
ids <- basename.pdb(pdbs$id)

anno <- pdb.annotate(ids)
unique(anno$source)

```

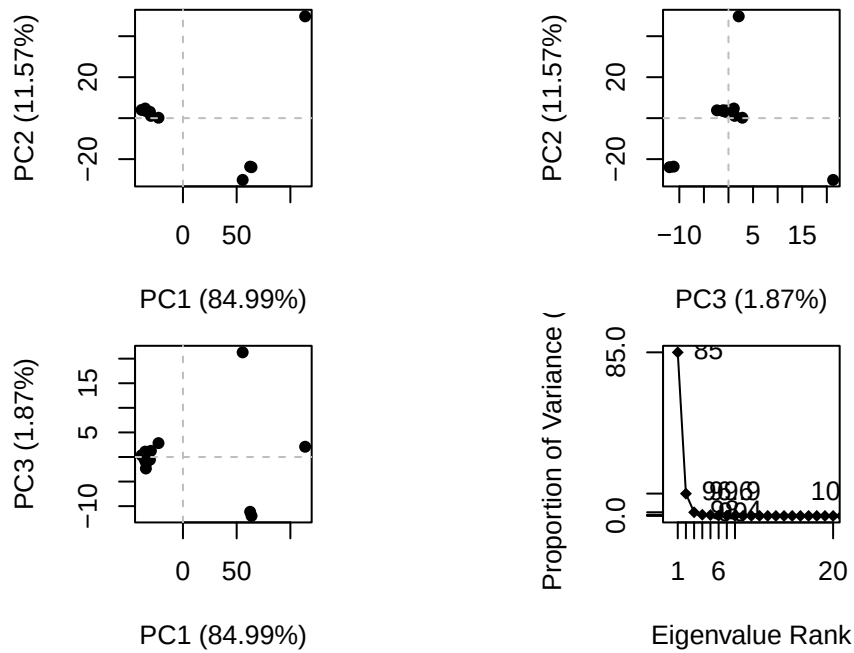
```

[1] "Escherichia coli"
[2] "Escherichia coli K-12"
[3] "Escherichia coli 0139:H28 str. E24377A"
[4] "Escherichia coli str. K-12 substr. MDS42"
[5] "Photobacterium profundum"
[6] "Burkholderia pseudomallei 1710b"
[7] "Francisella tularensis subsp. tularensis SCHU S4"

```

Principal component analysis

```
# Perform PCA
pc.xray <- pca(pdbbs)
plot(pc.xray)
```



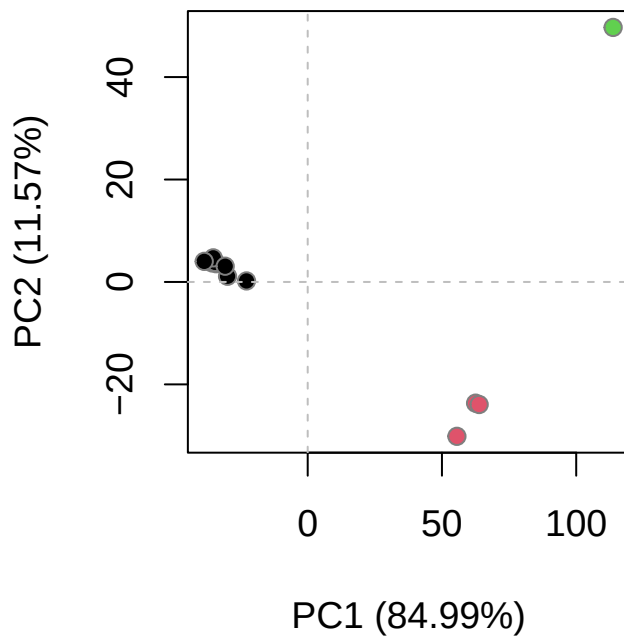
Function `rmsd()` will calculate all pairwise RMSD values of the structural ensemble. This facilitates clustering analysis based on the pairwise structural deviation:

```
# Calculate RMSD
rd <- rmsd(pdbbs)
```

Warning in `rmsd(pdbbs)`: No indices provided, using the 204 non NA positions

```
# Structure-based clustering
hc.rd <- hclust(dist(rd))
grps.rd <- cutree(hc.rd, k=3)

plot(pc.xray, 1:2, col="grey50", bg=grps.rd, pch=21, cex=1)
```

To visualize the major structural variations in the ensemble the function `mktrj()` can be used to generate a trajectory PDB file by interpolating along a give PC (eigenvector):

```
# Visualize first principal component
pc1 <- mktrj(pc.xray, pc=1, file="pc_1.pdb")
```

We can also view our results with the new `bio3dview` `view()` function:

```
##view.pca(pc.xray)
```

We can also plot our main PCA results with `ggplot`:

```
#Plotting results with ggplot2
library(ggplot2)
library(ggrepel)

df <- data.frame(PC1=pc.xray$z[,1],
                 PC2=pc.xray$z[,2],
                 col=as.factor(grps.rd),
                 ids=ids)

p <- ggplot(df) +
```

```

aes(PC1, PC2, col=col, label=ids) +
geom_point(size=2) +
geom_text_repel(max.overlaps = 20) +
theme(legend.position = "none")

```

p

